

Plantillas de implementación para aprendizaje supervisado

Notas de clase — patrones de implementación para el perceptrón simple y ADALINE

Jairo

Ingeniería de Sistemas, Universidad del Magdalena

Septiembre de 2026

Resumen

Estas notas presentan, en forma de plantilla reutilizable, el patrón de implementación común a los clasificadores lineales de la familia del perceptrón: el *perceptrón simple* de Rosenblatt y *ADALINE* (*Adaptive Linear Neuron*) de Widrow y Hoff. Ambos algoritmos comparten un mismo esquema de trabajo con datos —carga, segmentación, división en entrenamiento y prueba, estandarización, entrenamiento, evaluación y despliegue— y una misma familia de reglas de actualización de pesos por corrección de error. Se derivan ambas reglas a partir de sus respectivos criterios de optimalidad: la regla delta del perceptrón, del criterio de clasificación correcta, y la regla de ADALINE, del descenso de gradiente sobre una función de costo cuadrática diferenciable. Cada plantilla se acompaña de su código en Python, de la definición en prosa de sus componentes y del procedimiento para instanciarla sobre un conjunto de datos concreto. Se asume como prerequisite el álgebra lineal elemental (producto punto, norma), el cálculo diferencial de una variable y el manejo de `numpy` y `scikit-learn`.

Índice

1	Conceptos comunes	2
1.1	El problema de clasificación binaria supervisada	2
1.2	Entrada neta y regla de decisión	2
1.3	Corrección de error: el ancestro común de ambas reglas	3
1.4	El pipeline de datos: carga, limpieza y segmentación	3
1.5	Visualización de características y etiquetas	4
1.6	División de datos: entrenamiento y prueba	4
1.7	Estandarización	4
2	Perceptrón simple	5
2.1	Componentes	5
2.2	Plantilla	6
2.3	Modo de uso	7
3	ADALINE	8
3.1	Componentes	8
3.2	Plantilla	9
3.3	Modo de uso	11

4 Criterio de elección y errores frecuentes	11
4.1 Perceptrón frente a ADALINE	11
4.2 Elección de la tasa de aprendizaje	12
4.3 Errores frecuentes	12
4.4 Síntesis	13

1 Conceptos comunes

El perceptrón simple y ADALINE resuelven el mismo problema —clasificación binaria a partir de características numéricas— con la misma forma de decisión y con reglas de entrenamiento que comparten estructura algebraica. Esta sección fija, de una vez, el vocabulario y el pipeline de datos comunes a ambos; las Secciones 2 y 3 reutilizan estos términos sin volver a definirlos y se concentran en lo que distingue a cada algoritmo: la manera de calcular el error a partir del cual se corrigen los pesos.

1.1 El problema de clasificación binaria supervisada

Un problema de clasificación binaria supervisada parte de un conjunto de observaciones de las que se conoce tanto el valor de ciertas variables predictoras como la clase a la que pertenecen. El objetivo es construir una función que, a partir de las variables predictoras de una observación nueva, prediga su clase.

Definición 1.1 (Conjunto de datos etiquetado). Un conjunto de datos etiquetado para clasificación binaria es un par (X, y) donde $X \in \mathbb{R}^{N \times d}$ es una matriz cuyas N filas son las observaciones y cuyas d columnas son las características (*features*), y $y \in \{0, 1\}^N$ es el vector de etiquetas correspondiente, una por observación. Se escribe $x^{(i)} \in \mathbb{R}^d$ para la i -ésima fila de X y $y^{(i)}$ para su etiqueta.

1.2 Entrada neta y regla de decisión

Tanto el perceptrón como ADALINE clasifican mediante una función lineal de las características, seguida de un umbral.

Definición 1.2 (Entrada neta). La entrada neta de una observación $x \in \mathbb{R}^d$ bajo un vector de pesos $w \in \mathbb{R}^d$ y un sesgo $b \in \mathbb{R}$ es el escalar

$$z = w^\top x + b = \sum_{j=1}^d w_j x_j + b.$$

El par (w, b) constituye el *modelo* en el sentido en que ambos algoritmos entienden ese término: es lo único que el entrenamiento modifica y lo único que la predicción necesita.

Definición 1.3 (Regla de decisión). La regla de decisión asociada a un modelo (w, b) es la función escalón

$$\hat{y} = \varphi(z) = \begin{cases} 1 & \text{si } z \geq 0 \\ 0 & \text{si } z < 0 \end{cases}$$

que convierte la entrada neta en una etiqueta predicha $\hat{y} \in \{0, 1\}$.

Observación 1.4. La regla de decisión es la misma para los dos algoritmos: ambos son *clasificadores lineales* y la frontera de decisión que producen, $\{x \in \mathbb{R}^d : w^\top x + b = 0\}$, es un hiperplano en ambos casos. Lo que distingue al perceptrón de ADALINE no es cómo se clasifica una vez fijados los pesos, sino cómo se ajustan los pesos durante el entrenamiento, que es el tema de las Secciones 2 y 3.

1.3 Corrección de error: el ancestro común de ambas reglas

Antes de derivar cada regla por separado conviene señalar el principio que comparten. Ambos algoritmos ajustan w y b de manera iterativa, y en ambos casos la actualización tiene la forma

$$\Delta w = \eta \cdot (\text{error}) \cdot x, \quad \Delta b = \eta \cdot (\text{error}),$$

donde $\eta > 0$ es la *tasa de aprendizaje* y el error se calcula comparando la etiqueta real con una salida producida por el modelo. La diferencia entre los dos algoritmos, desarrollada en detalle en las secciones siguientes, está enteramente en qué salida se usa para calcular ese error: el perceptrón compara la etiqueta real con la predicción *discreta* $\hat{y} = \varphi(z)$; ADALINE compara la etiqueta real con la entrada neta *continua* z , antes de aplicar el escalón. Esa única diferencia de un paso es la que hace a ADALINE diferenciable y al perceptrón no.

1.4 El pipeline de datos: carga, limpieza y segmentación

La instanciación de cualquiera de las dos plantillas comienza por preparar los datos. Esta etapa es independiente del algoritmo de clasificación que se use después, y se resume en tres pasos.

1. **Carga y selección de características.** Se carga el conjunto de datos y se seleccionan las d columnas que se usarán como características, descartando las que no sean relevantes para el problema y verificando la ausencia de valores faltantes (NaN) en las columnas elegidas.
2. **Segmentación a un problema binario.** El perceptrón y ADALINE, en su formulación clásica, requieren exactamente dos clases. Si el conjunto de datos original tiene más de dos, se aplica una máscara booleana sobre las etiquetas para retener solo las dos que interesan.
3. **Verificación del resultado.** Se comprueba la forma de X y y resultantes y que ambas clases queden representadas.

```
# 1. Carga y seleccion de características (ejemplo generico; sustituir
# dataset_loader por la fuente real de datos, p. ej. sklearn.datasets)
X = dataset.data[:, [0, 1]] # d columnas elegidas como features
y = dataset.target # etiquetas originales

# 2. Segmentacion a un problema binario
mask = y < 2 # condicion que retiene solo dos clases
X = X[mask]
y = y[mask]

# 3. Verificacion
assert X.shape[0] == y.shape[0]
assert set(y) == {0, 1}
```

Observación 1.5. La máscara del paso 2 opera sobre y y debe aplicarse idénticamente sobre X : son las dos caras del mismo filtro, y aplicarlas por separado con condiciones distintas desalinea las observaciones con sus etiquetas sin producir ningún error visible en tiempo de ejecución.

1.5 Visualización de características y etiquetas

Antes de entrenar conviene inspeccionar visualmente si las clases son separables con las características elegidas. Con dos características, un diagrama de dispersión coloreado por clase es la herramienta estándar.

```
import matplotlib.pyplot as plt

for clase in sorted(set(y)):
    plt.scatter(X[y == clase, 0], X[y == clase, 1], label=f"clase {clase}")
plt.xlabel("característica 0")
plt.ylabel("característica 1")
plt.legend()
plt.show()
```

1.6 División de datos: entrenamiento y prueba

El modelo se ajusta sobre una parte de los datos y se evalúa sobre una parte distinta que no participó del ajuste, de modo que la precisión medida refleje la capacidad de generalización del modelo y no su capacidad de memorizar los datos de entrenamiento.

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    train_size=0.7, test_size=0.3, # proporciones a definir segun el caso
    random_state=1, # fija la particion para reproducibilidad
    stratify=y # conserva la proporcion de clases en ambos conjuntos
)
```

Observación 1.6. El argumento `stratify=y` no es opcional cuando las clases están desbalanceadas: sin él, `train_test_split` reparte las filas al azar y una partición desafortunada puede dejar una clase subrepresentada en el conjunto de prueba, lo que vuelve engañosa la precisión medida sobre ese conjunto.

1.7 Estandarización

Definición 1.7 (Estandarización). La estandarización de una característica x_j respecto de un conjunto de entrenamiento es la transformación

$$x'_j = \frac{x_j - \bar{x}_j}{\sigma_j},$$

donde $\bar{x}_j$ y σ_j son, respectivamente, la media y la desviación estándar de x_j calculadas *únicamente* sobre el conjunto de entrenamiento. El resultado es una característica de media cero y varianza unitaria en el conjunto de entrenamiento.

```
from sklearn.preprocessing import StandardScaler

est = StandardScaler()
X_train_std = est.fit_transform(X_train) # calcula xbarra, sigma y transforma
X_test_std = est.transform(X_test) # aplica los MISMOS xbarra, sigma
```

Observación 1.8 (El error más frecuente de esta etapa). El conjunto de prueba se transforma con `transform`, nunca con `fit_transform`. Si el conjunto de prueba recalculara su propia media y desviación estándar, estaría usando información —la distribución de los datos de prueba— que en un escenario real todavía no existe en el momento de entrenar; el resultado es una fuga de información (*data leakage*) que hace que la precisión medida sobre el conjunto de prueba deje de ser un estimador honesto del desempeño del modelo sobre datos nuevos.

Observación 1.9 (Por qué importa para estos dos algoritmos en particular). La estandarización no es un paso cosmético para el perceptrón y ADALINE: ambos actualizan w proporcionalmente a x (Sección 1.2), de modo que una característica con una escala mucho mayor que las demás domina la actualización del peso correspondiente y hace que el descenso avance de forma desigual en cada dirección. La Sección 4.2 retoma esta observación al discutir la elección de η .

2 Perceptrón simple

El perceptrón simple, introducido por Rosenblatt [1], es el más antiguo de los dos algoritmos y el más restrictivo en su criterio de actualización: solo corrige los pesos cuando la predicción actual es incorrecta, y lo hace desplazándolos en la dirección que habría producido la clasificación correcta para esa observación.

2.1 Componentes

Definición 2.1 (Regla delta del perceptrón). Dada una observación (x, y) y un modelo (w, b) , la regla delta del perceptrón actualiza los pesos según

$$\Delta w = \eta(y - \hat{y})x, \quad \Delta b = \eta(y - \hat{y}),$$

donde $\hat{y} = \varphi(w^\top x + b)$ es la predicción *discreta* del modelo actual bajo la regla de decisión de la Sección 1.2 y $\eta > 0$ es la tasa de aprendizaje.

Observación 2.2. Como $y, \hat{y} \in \{0, 1\}$, el factor $(y - \hat{y})$ solo toma los valores -1 , 0 o 1 : vale 0 exactamente cuando la predicción ya es correcta, caso en el que la Definición 2.1 no modifica el modelo. La regla delta es, en este sentido, puramente correctiva: no hay actualización sin error de clasificación.

Ejemplo 2.3. Sea $x = -1$ (una sola característica), $y = 1$, $w = 0.5$, $b = 0$ y $\eta = 0.1$. La entrada neta es

$$z = wx + b = (0.5)(-1) + 0 = -0.5,$$

de modo que $\hat{y} = \varphi(-0.5) = 0$: la observación queda mal clasificada. El error es $y - \hat{y} = 1 - 0 = 1$, y la Definición 2.1 produce

$$\Delta w = \eta(y - \hat{y})x = (0.1)(1)(-1) = -0.1, \quad \Delta b = \eta(y - \hat{y}) = (0.1)(1) = 0.1.$$

Los pesos actualizados son $w' = 0.5 - 0.1 = 0.4$ y $b' = 0 + 0.1 = 0.1$.

Teorema 2.4 (Convergencia del perceptrón). *Si las dos clases de un conjunto de entrenamiento son linealmente separables, el algoritmo del perceptrón, aplicado repetidamente sobre las observaciones del conjunto, encuentra en un número finito de pasos un vector de pesos que las clasifica correctamente a todas.*

Demostración. Véase la demostración original de Novikoff [3], basada en acotar simultáneamente por arriba y por abajo el crecimiento de $\|w\|$ en función del número de correcciones. $\square$

Observación 2.5. El recíproco de la garantía de convergencia es la limitación práctica del algoritmo: si las clases *no* son linealmente separables, el perceptrón nunca deja de corregir pesos y el número de iteraciones (`max_iter`) actúa como único criterio de parada, sin que ese límite indique convergencia real. La Sección 4.1 retoma esta limitación al comparar el perceptrón con ADALINE.

2.2 Plantilla

La instanciación del perceptrón simple reutiliza el pipeline de datos de la Sección 1.4 hasta la estandarización inclusive; a partir de ahí, los pasos propios del algoritmo son los siguientes.

```
from sklearn.linear_model import Perceptron
from sklearn.metrics import accuracy_score
import joblib

# Parametrización del modelo
mod = Perceptron(
    eta0=0.1, # tasa de aprendizaje eta, en (0, 1]
    max_iter=100, # tope de iteraciones (epocas) sobre el conjunto de entrenamiento
    n_jobs=1, # núcleos de CPU a usar
    random_state=1 # semilla para la inicialización y el orden de recorrido
)

# Entrenamiento
mod.fit(X_train_std, y_train)

# Predicción y precisión
y_pred = mod.predict(X_test_std)
accur = accuracy_score(y_test, y_pred)

# Despliegue a producción
joblib.dump(mod, "perceptron_model.joblib")
# en producción: mod = joblib.load("perceptron_model.joblib")
```

La visualización de las regiones de decisión aprendidas se obtiene evaluando la regla de decisión del modelo sobre una malla de puntos que cubre el plano de las dos características y coloreando cada punto de la malla según la clase que el modelo le asignaría.

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.colors import ListedColormap

def plot_decision_regions(X, y, clasificador, resolution=0.02):
    colores = ("#1f77b4", "#d62728")
    cmap = ListedColormap(colores[:len(np.unique(y))])

    x1_min, x1_max = X[:, 0].min() - 1, X[:, 0].max() + 1
    x2_min, x2_max = X[:, 1].min() - 1, X[:, 1].max() + 1
    xx1, xx2 = np.meshgrid(np.arange(x1_min, x1_max, resolution),
                           np.arange(x2_min, x2_max, resolution))
    Z = clasificador.predict(np.array([xx1.ravel(), xx2.ravel()]).T)
```

```

Z = Z.reshape(xx1.shape)

plt.contourf(xx1, xx2, Z, alpha=0.3, cmap=cmap)
for idx, clase in enumerate(np.unique(y)):
    plt.scatter(X[y == clase, 0], X[y == clase, 1],
                label=f"clase {clase}", edgecolor="black")
plt.legend()

```

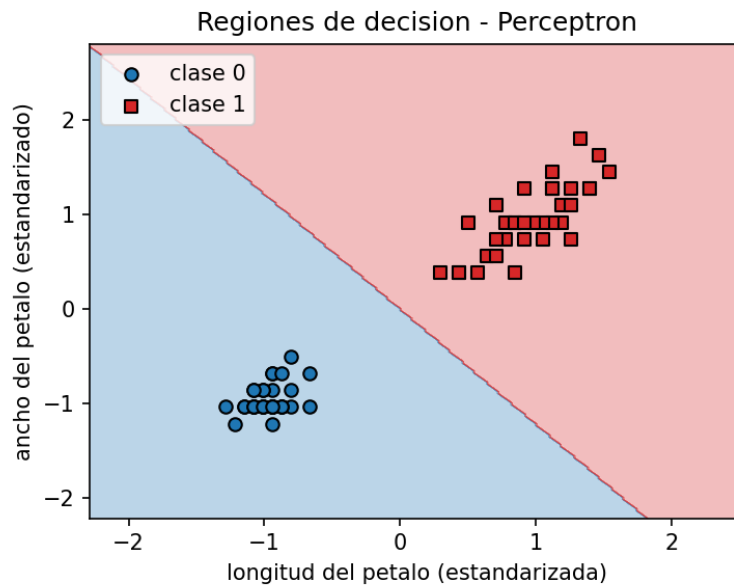


Figura 1: Regiones de decisión aprendidas por **Perceptron** de **scikit-learn** sobre dos características de un conjunto de datos separable, tras estandarización. La frontera es el hiperplano (aquí, una recta) $w^T x + b = 0$ de la Sección 1.2.

2.3 Modo de uso

1. Ejecutar el pipeline de datos de la Sección 1.4 (carga, segmentación, división, estandarización) sobre el conjunto de datos del ejercicio.
2. Elegir η (`eta0`) en $(0, 1]$: un valor típico de partida es 0.1; la Sección 4.2 da el criterio para ajustarlo.
3. Fijar `max_iter` como una cota superior generosa de épocas y `random_state` para que el experimento sea reproducible.
4. Entrenar con `fit` sobre `X_train_std`, `y_train` y predecir con `predict` sobre `X_test_std`.
5. Calcular `accuracy_score(y_test, y_pred)`. Si la precisión es baja y el número de errores de entrenamiento no llegó a cero dentro de `max_iter`, sospechar que las clases no son linealmente separables (Observación 2.5) antes de aumentar `max_iter` indefinidamente.
6. Visualizar las regiones de decisión con `plot_decision_regions` cuando el número de características usadas sea dos, para inspeccionar cualitativamente el hiperplano aprendido.

7. Serializar el modelo entrenado con `joblib.dump` únicamente después de validar la precisión sobre el conjunto de prueba.

3 ADALINE

ADALINE (*Adaptive Linear Neuron*), introducido por Widrow y Hoff [2], comparte con el perceptrón la entrada neta y la regla de decisión de la Sección 1.2, pero cambia el criterio de actualización: en vez de corregir los pesos únicamente ante un error de clasificación discreto, ADALINE minimiza una función de costo continua y diferenciable de la entrada neta, lo que permite derivar su regla de actualización por descenso de gradiente en vez de postularla directamente.

3.1 Componentes

Definición 3.1 (Función de costo de ADALINE). Dado un conjunto de entrenamiento $\{(x^{(i)}, y^{(i)})\}_{i=1}^N$ codificado con $y^{(i)} \in \{-1, +1\}$, la función de costo de ADALINE bajo un modelo (w, b) es la suma de errores cuadráticos entre la etiqueta real y la entrada neta *sin* umbralizar,

$$J(w, b) = \frac{1}{2} \sum_{i=1}^N \left(y^{(i)} - z^{(i)} \right)^2, \quad z^{(i)} = w^\top x^{(i)} + b.$$

Observación 3.2. La codificación de las etiquetas como $\{-1, +1\}$, en vez de $\{0, 1\}$, es una convención que hace a J simétrica respecto del umbral de decisión $z = 0$: da al modelo la misma penalización relativa por acercarse por exceso o por defecto a cada clase. La regla de decisión final (Sección 1.2) se expresa entonces como $\hat{y} = +1$ si $z \geq 0$ y $\hat{y} = -1$ en caso contrario, y se traduce de vuelta a las etiquetas originales $\{0, 1\}$ únicamente para reportarla al usuario.

Proposición 3.3 (Gradiente de J). *El gradiente de J respecto de los pesos y el sesgo es*

$$\frac{\partial J}{\partial w} = - \sum_{i=1}^N \left(y^{(i)} - z^{(i)} \right) x^{(i)}, \quad \frac{\partial J}{\partial b} = - \sum_{i=1}^N \left(y^{(i)} - z^{(i)} \right).$$

Demostración. Se desarrolla la derivada para un peso w_j arbitrario; el caso de b es análogo y se omite por ser formalmente idéntico con $\partial z^{(i)} / \partial b = 1$. Puesto que $z^{(i)} = w^\top x^{(i)} + b$ depende de w_j únicamente a través del término $w_j x_j^{(i)}$, la regla de la cadena da, para cada sumando de J ,

$$\begin{aligned} \frac{\partial}{\partial w_j} \left[\frac{1}{2} \left(y^{(i)} - z^{(i)} \right)^2 \right] &= \left(y^{(i)} - z^{(i)} \right) \cdot \frac{\partial}{\partial w_j} \left(y^{(i)} - z^{(i)} \right) \\ &= \left(y^{(i)} - z^{(i)} \right) \cdot \left(- \frac{\partial z^{(i)}}{\partial w_j} \right) \\ &= - \left(y^{(i)} - z^{(i)} \right) x_j^{(i)}. \end{aligned}$$

Sumando sobre las N observaciones y agrupando en notación vectorial,

$$\frac{\partial J}{\partial w} = \sum_{i=1}^N \left[- \left(y^{(i)} - z^{(i)} \right) x^{(i)} \right] = - \sum_{i=1}^N \left(y^{(i)} - z^{(i)} \right) x^{(i)},$$

que es la expresión enunciada. □

Definición 3.4 (Regla de ADALINE por descenso de gradiente). La regla de actualización de ADALINE es un paso de descenso de gradiente sobre J : se avanza en la dirección opuesta al gradiente de la Proposición 3.3, escalada por la tasa de aprendizaje η ,

$$\Delta w = -\eta \frac{\partial J}{\partial w} = \eta \sum_{i=1}^N (y^{(i)} - z^{(i)}) x^{(i)}, \quad \Delta b = \eta \sum_{i=1}^N (y^{(i)} - z^{(i)}).$$

Observación 3.5. Comparando la Definición 3.4 con la regla delta del perceptrón (Definición 2.1) se ve la diferencia exacta entre ambos algoritmos: la misma forma $\eta \cdot (\text{error}) \cdot x$, pero el perceptrón calcula el error con la predicción *umbralizada* $\hat{y} = \varphi(z)$ de una sola observación a la vez, mientras que ADALINE lo calcula con la entrada neta *sin umbralizar* z , acumulada sobre todo el conjunto de entrenamiento antes de actualizar (descenso de gradiente *por lotes*). Esa diferencia de un paso—umbralizar antes o después de comparar— es la que hace a J diferenciable y permite aplicar cálculo para obtenerla, en vez de postular la regla directamente como se hizo para el perceptrón.

Ejemplo 3.6. Sea un conjunto de dos observaciones con una sola característica, $x^{(1)} = 1$, $x^{(2)} = 2$, con etiquetas $y^{(1)} = -1$, $y^{(2)} = +1$, y modelo inicial $w = 0$, $b = 0$, con $\eta = 0.1$. La entrada neta de ambas observaciones bajo el modelo inicial es $z^{(1)} = z^{(2)} = 0$, de modo que

$$J(0, 0) = \frac{1}{2} [(-1 - 0)^2 + (1 - 0)^2] = \frac{1}{2}(1 + 1) = 1.$$

Los errores son $y^{(1)} - z^{(1)} = -1$ y $y^{(2)} - z^{(2)} = 1$, y la Definición 3.4 produce

$$\Delta w = \eta \sum_{i=1}^2 (y^{(i)} - z^{(i)}) x^{(i)} = (0.1)[(-1)(1) + (1)(2)] = (0.1)(1) = 0.1, \quad \Delta b = (0.1)(-1 + 1) = 0.$$

Con $w' = 0.1$ y $b' = 0$, las nuevas entradas netas son $z^{(1)'} = 0.1$ y $z^{(2)'} = 0.2$, y el costo baja a

$$J(0.1, 0) = \frac{1}{2} [(-1 - 0.1)^2 + (1 - 0.2)^2] = \frac{1}{2}(1.21 + 0.64) = 0.925 < 1 = J(0, 0),$$

como debe ocurrir tras un paso en la dirección de descenso.

3.2 Plantilla

ADALINE no está disponible como clase de `scikit-learn` porque el paquete no ofrece un clasificador lineal con activación identidad entrenado por mínimos cuadrados en línea; se implementa como una clase propia que expone la misma interfaz `fit/predict` de `scikit-learn` (heredando de `BaseEstimator` y `ClassifierMixin`) para integrarse sin cambios en el resto del pipeline de la Sección 1.4.

```
import numpy as np
from sklearn.base import BaseEstimator, ClassifierMixin

class AdalineGD(BaseEstimator, ClassifierMixin):
    """ADALINE entrenado por descenso de gradiente por lotes."""

    def __init__(self, eta0=0.01, max_iter=100, random_state=1):
        self.eta0 = eta0
        self.max_iter = max_iter
```

```

        self.random_state = random_state

    def net_input(self, X):
        return np.dot(X, self.coef_.T).ravel() + self.intercept_

    def activation(self, z):
        return z # identidad: se usa TAL CUAL en el entrenamiento

    def fit(self, X, y):
        # traduce las etiquetas originales a {-1, +1} para el entrenamiento
        self.classes_ = np.unique(y)
        y_pm1 = np.where(y == self.classes_[1], 1, -1)

        rgen = np.random.RandomState(self.random_state)
        self.coef_ = rgen.normal(loc=0.0, scale=0.01, size=(1, X.shape[1]))
        self.intercept_ = np.zeros(1)
        self.cost_ = []

        for _ in range(self.max_iter):
            z = self.net_input(X)
            output = self.activation(z)
            errores = y_pm1 - output # Definicion 3.4
            self.coef_ += self.eta0 * (errores @ X).reshape(1, -1)
            self.intercept_ += self.eta0 * errores.sum()
            self.cost_.append((errores ** 2).sum() / 2.0) # Definicion 3.1
        return self

    def predict(self, X):
        z = self.net_input(X)
        return np.where(z >= 0.0, self.classes_[1], self.classes_[0])

```

A partir de aquí el modelo se usa exactamente como el del perceptrón: se parametriza, se entrena, se evalúa y se despliega con las mismas funciones del pipeline común.

```

from sklearn.metrics import accuracy_score
import joblib

mod = AdalineGD(eta0=0.01, max_iter=50, random_state=1)
mod.fit(X_train_std, y_train)

y_pred = mod.predict(X_test_std)
accur = accuracy_score(y_test, y_pred)

joblib.dump(mod, "adaline_model.joblib")

```

La curva de costo por época, ausente en el perceptrón porque este no minimiza una función continua, es la herramienta diagnóstica propia de ADALINE: debe decrecer monótonamente hasta estabilizarse si η está bien elegido (Sección 4.2).

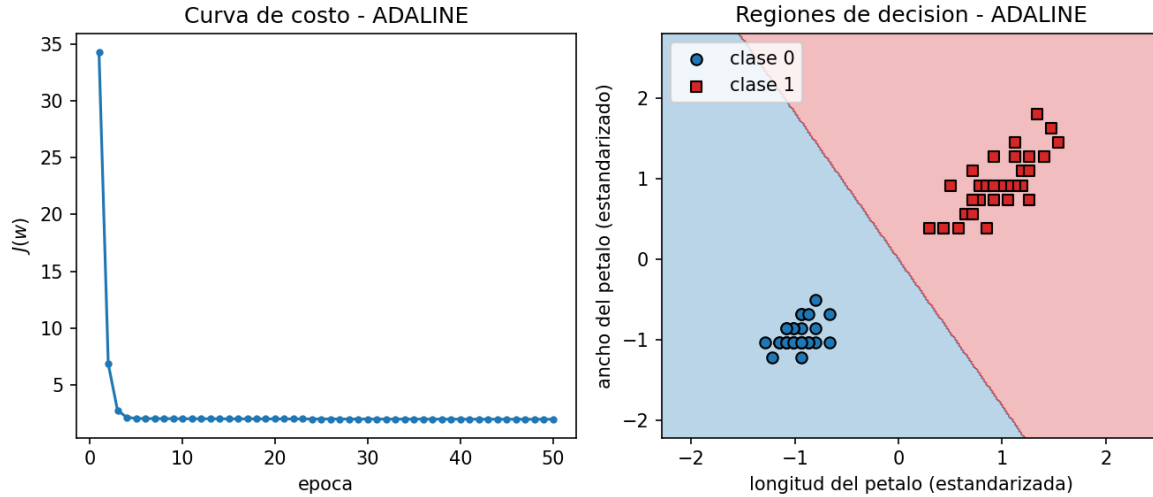


Figura 2: Izquierda: curva de costo $J(w)$ de la Definición 3.1 por época de entrenamiento, decreciente y decreciendo cada vez más lento a medida que el modelo se acerca al mínimo. Derecha: regiones de decisión aprendidas sobre el mismo conjunto de datos y las mismas dos características que la Figura 1.

3.3 Modo de uso

1. Ejecutar el mismo pipeline de datos de la Sección 1.4: carga, segmentación, división y estandarización son idénticas a las del perceptrón.
2. Instanciar `AdalineGD` con `eta0` y `max_iter`; a diferencia del perceptrón, aquí conviene partir de un η pequeño (por ejemplo, 0.01) porque la actualización se acumula sobre *todas* las observaciones antes de aplicarse, y no una a la vez.
3. Entrenar con `fit` y examinar `mod.cost_`: si la sucesión no decrece de forma sostenida, reducir η (Sección 4.2) antes de cambiar cualquier otro elemento del pipeline.
4. Predecir con `predict` sobre el conjunto de prueba estandarizado y calcular `accuracy_score` exactamente como con el perceptrón.
5. Graficar la curva de costo junto con las regiones de decisión: la primera diagnostica el entrenamiento, la segunda diagnostica el resultado.
6. Serializar con `joblib.dump` una vez validada la precisión sobre el conjunto de prueba.

4 Criterio de elección y errores frecuentes

4.1 Perceptrón frente a ADALINE

La Observación 3.5 ya señaló la diferencia estructural entre ambas reglas de actualización; de ella se derivan las consecuencias prácticas que orientan cuál algoritmo usar.

Observación 4.1 (Garantía de convergencia frente a garantía de un mínimo). El perceptrón converge en un número finito de pasos únicamente si las clases son linealmente separables (Observación 2.5);

si no lo son, el algoritmo oscila indefinidamente y no hay indicio, dentro del propio entrenamiento, de qué tan cerca está de una buena solución. ADALINE, en cambio, minimiza una función de costo J que es una forma cuadrática de (w, b) y por lo tanto convexa: el descenso de gradiente converge siempre a su mínimo global (con η adecuado), sea o no separable el conjunto de datos. Ese mínimo no necesariamente clasifica bien todas las observaciones cuando las clases se solapan, pero su existencia y alcanzabilidad no dependen de la separabilidad.

Observación 4.2 (Relevancia de ADALINE más allá de sí misma). ADALINE no es solo un segundo algoritmo de la misma familia: al reemplazar el criterio de clasificación correcta por la minimización de una función de costo diferenciable, introduce el esquema —función de costo continua, gradiente, descenso— sobre el que se construyen la regresión logística y, por extensión, el entrenamiento por retropropagación de las redes neuronales multicapa [4]. El paso de un umbral discreto a una activación continua durante el entrenamiento, que aquí distingue a dos algoritmos de un curso introductorio, es el mismo paso conceptual que separa un perceptrón multicapa entrenable de una red de perceptrones sin regla de aprendizaje diferenciable.

4.2 Elección de la tasa de aprendizaje

Observación 4.3 (η demasiado grande). Un η excesivo hace que cada paso sobrepase el mínimo de J en vez de acercarse a él; en ADALINE esto es diagnosticable directamente en la curva de costo, que en vez de decrecer crece sin límite de una época a la siguiente. Sobre el ejemplo de la Sección 3, tomar $\eta = 1.5$ en lugar de $\eta = 0.01$ produce, en las primeras épocas, un costo que pasa de un valor del orden de 10^1 a valores del orden de 10^6 , 10^{10} y mayores en las épocas siguientes: la sucesión diverge en vez de converger.

Observación 4.4 (η demasiado pequeño). Un η insuficiente hace que el algoritmo avance en la dirección correcta pero a pasos tan cortos que, dentro del número de épocas disponible, el costo apenas se reduce: sobre el mismo ejemplo, $\eta = 0.0001$ deja, tras el mismo número de épocas que $\eta = 0.01$, un costo varias veces mayor que el alcanzado con la tasa bien elegida, sin que el entrenamiento haya fallado —solo es más lento de lo necesario—.

Nota 4.5. La estandarización de la Sección 1.7 interactúa directamente con esta elección: al llevar todas las características a la misma escala, permite usar un único η para todas las componentes de w sin que unas se actualicen demasiado rápido y otras demasiado lento.

4.3 Errores frecuentes

Observación 4.6 (Fuga de información en la estandarización). Ya señalado en la Observación 1.8: ajustar el `StandardScaler` sobre el conjunto de prueba, en vez de solo transformarlo con los parámetros del conjunto de entrenamiento, invalida la precisión medida.

Observación 4.7 (Confundir la salida de entrenamiento con la predicción). En ADALINE, `activation(z)` devuelve la entrada neta sin modificar durante el entrenamiento (Definición 3.1), pero `predict` debe seguir aplicando la regla de decisión con umbral (Sección 1.2) para devolver una etiqueta de clase. Omitir ese umbral en `predict` y devolver directamente la salida continua de `activation` produce un objeto que no es un clasificador y que `accuracy_score` no puede evaluar de forma consistente.

Observación 4.8 (Interpretar `max_iter` como evidencia de convergencia). Que `Perceptron` o `AdalineGD` completen sus `max_iter` épocas sin lanzar ningún error no significa que hayan convergido. En el perceptrón, la señal real de convergencia es que el número de errores de clasificación sobre el conjunto de entrenamiento llegue a cero antes de agotar las épocas (Observación 2.5); en ADALINE, es que

la curva de costo se estabilice (Sección 4.2). Ninguna de las dos señales se obtiene automáticamente del hecho de que el bucle de entrenamiento haya terminado.

Observación 4.9 (Elegir el algoritmo por la biblioteca disponible). Que `scikit-learn` ofrezca `Perceptron` y no ofrezca `ADALINE` no es un argumento para preferir el primero: la elección entre ambos depende de si el conjunto de datos es linealmente separable y de si se necesita una función de costo diferenciable para razonar sobre la calidad del ajuste durante el entrenamiento (Sección 4.1), no de la disponibilidad de una implementación ya hecha.

4.4 Síntesis

Ambas plantillas comparten el pipeline de datos, la entrada neta, la regla de decisión y la forma general $\eta \cdot (\text{error}) \cdot x$ de su actualización de pesos. Se diferencian en un único punto, del que se derivan todas sus demás diferencias prácticas: el perceptrón calcula el error con la predicción ya umbralizada de una observación a la vez, mientras que ADALINE lo calcula con la entrada neta continua, acumulada sobre todo el conjunto de entrenamiento, lo que convierte su actualización en un paso de descenso de gradiente sobre una función de costo diferenciable. Instanciar cualquiera de las dos plantillas consiste en preparar los datos según la Sección 1.4, parametrizar el modelo correspondiente, entrenarlo, verificar su convergencia con el diagnóstico propio de cada algoritmo y evaluar su precisión sobre datos que no participaron del ajuste.

Referencias

- [1] F. Rosenblatt, *The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain*, Psychological Review, vol. 65, n.º 6, 1958.
- [2] B. Widrow y M. E. Hoff, *Adaptive Switching Circuits*, IRE WESCON Convention Record, 1960.
- [3] A. B. J. Novikoff, *On Convergence Proofs on Perceptrons*, Proceedings of the Symposium on the Mathematical Theory of Automata, vol. 12, 1962.
- [4] S. Raschka y V. Mirjalili, *Python Machine Learning*, 3.^a ed., Packt Publishing, 2019.
- [5] scikit-learn developers, *Perceptron — scikit-learn documentation*, https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.Perceptron.html.